LSGI3245 – Geospatial Database Management and Design

Lecture 3 - SQL (I)

Instructor: Yue YU

Department of Land Surveying and Geo-Informatics,

The Hong Kong Polytechnic University

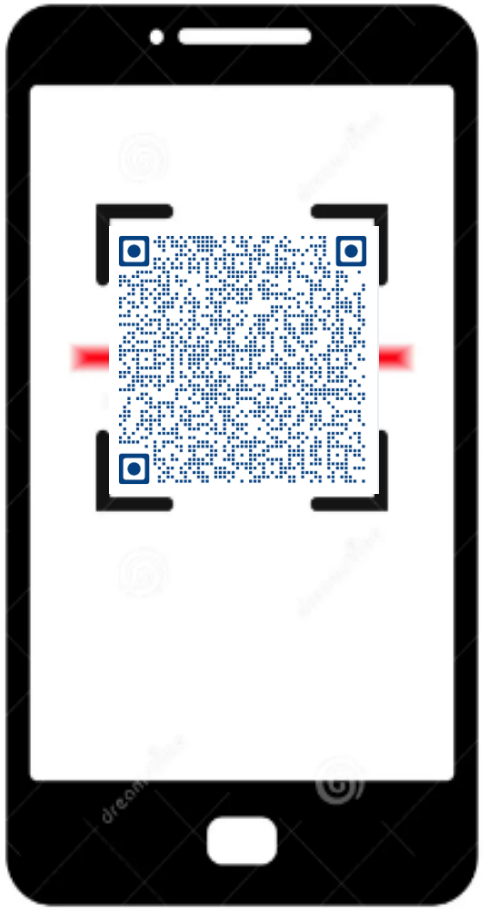
Tentative Schedule

Week	Lecture (Tuesday)		Lab (Wednesday)	
1	1-Sept	Course Introduction	Lab Introduction and Software Installation	2-Sept
2	8-Sept	Relational Model	Introduction to PostgreSQL (Part 1)	9-Sept
3	15-Sept	SQL (Part 1) Release Information for Final Project	Introduction to PostgreSQL (Part 2)	16-Sept
4	22-Sept	SQL (Part 2)	Query and Data Manipulation Using SQL (Part 1)	23-Sept
5	29-Sept	Intermediate SQL	Query and Data Manipulation Using SQL (Part 2)	30-Sept
6	6-Oct	Database Design	Introduction to PostGIS	7-Oct
7	13-Oct	Spatial Database & PostGIS	PostGIS: Spatial Datatype and Relationships	14-Oct
8	20-Oct	Test 1	PostGIS: Spatial Queries	21-Oct
9	27-Oct	PostGIS & Spatial Queries	Students Work on Final Projects (Part 1)	28-Oct
10	3-Nov	Spatial Storage and Indexing	Students Work on Final Projects (Part 2)	4-Nov
11	10-Nov	Test 2	Students Compile Final Project Reports (Part 1)	11-Nov
12	17-Nov	Project Presentation (Part 1)	Students Compile Final Project Reports (Part 2)	18-Nov
13	24-Nov	Project Presentation (Part 2)	Summary and Outlook	25-Nov

Final Project

Final Project Grouping

- 5 students per group (one as group leader)
  - Note: The assessment is based on the group as an integral part. The contributions of all the group members should be clearly stated in the oral presentation and the final written report, which are also considered for the final assessment of each group member.
- Give the group a good name
- Submit the list of group members via Google Spreadsheet
- **DDL**: by Sep 22, Tuesday (Week 4)
- If you have any queries about grouping, pls contact our TA for help: **Mr. Zhanpeng Zhang** (zhanpeng-2026.zhang@connect.polyu.hk) and **Mr. Feifan Lin** (Feifa.lin@polyu.edu.hk)

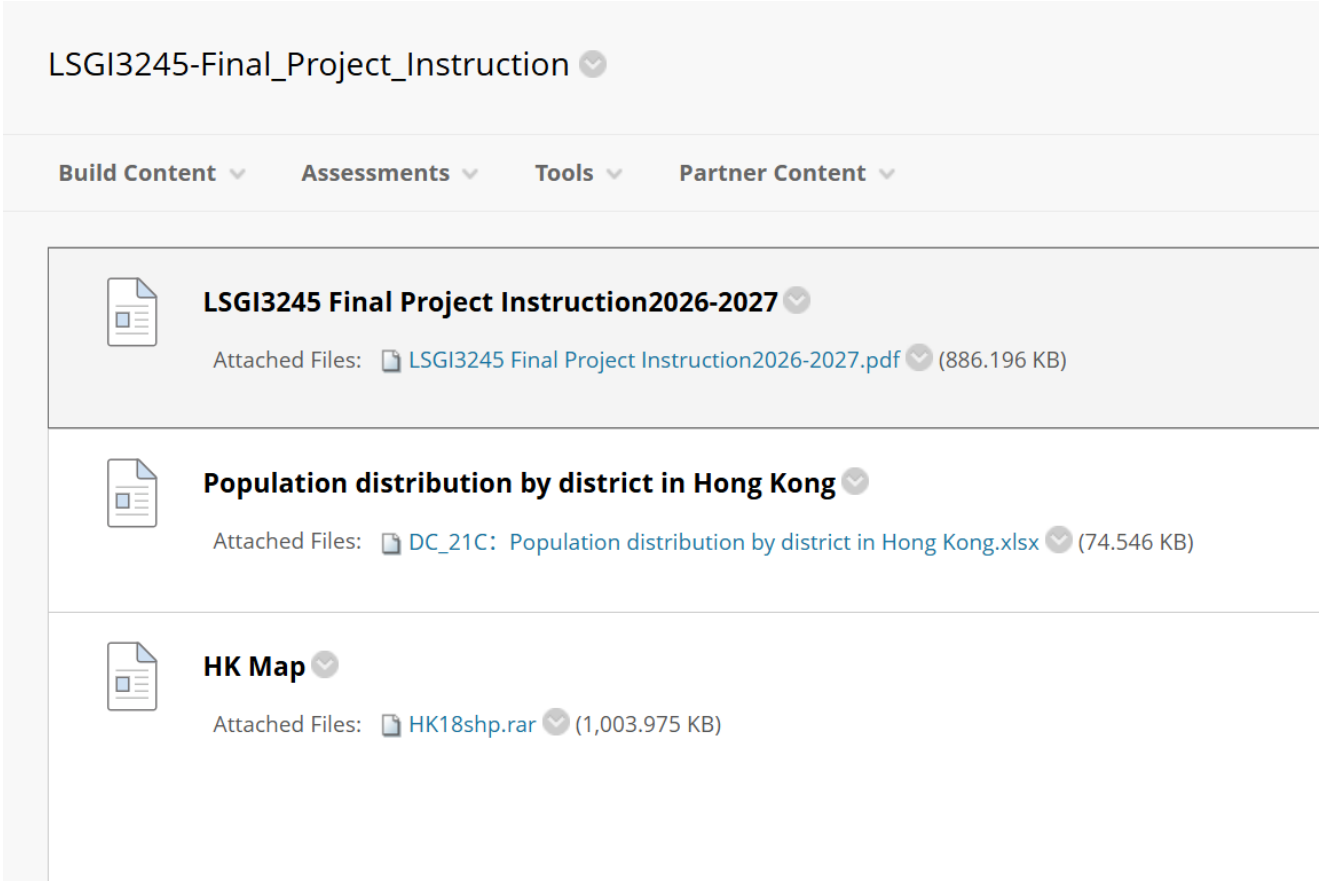


Please Scan the QR code to fill the group information

Overview of Final Project

- Weight: 15%
- Group: 5 students per group
- Deliverables: an oral presentation (10 min/group), a written report (with supplement SQL code)
- Task:
  - To incorporate both spatial and non-spatial information into a database design, and then implement them as a relational database; also demonstrate the mastery of spatial and non-spatial queries/operations over the database created.
- Instruction: Blackboard-Content/LSGI3245-Final_Project_Instruction

Final Project Instruction



Recap



- What is a **relational model**?
  - The relational model is based on a collection of tables. The user of the database system may query these tables, insert new tuples, delete tuples, and update tuples.
  - In the relational model, the term relation is used to refer to a table.

- Important concepts about relational model
  - Relation schema & instance
  - Attributes
  - Keys

Recap (Cont.)



- Relational Query Languages
  - Relational algebra
- Basic syntax of SQL
  - create table
  - select
  - from
  - where

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{salary >= 85000}(instructor)$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID,\ salary}(instructor)$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
x (Cartesian Product)	*instructor* x *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
∪ (Union)	$\Pi_{name}(instructor) \cup \Pi_{name}(student)$
	Output the union of tuples from the *two* input relations.
– (Set Difference)	$\Pi_{name}(instructor) - \Pi_{name}(student)$
	Output the set difference of tuples from the two input relations.
⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

Source: http://www.db-book.com/

Review of Key Concepts

Organization of the Concepts

Users

Application interface

communicate

PostgreSQL

Database system

DB languages

DBMS

abstraction

Data abstraction

implementation

Database engines

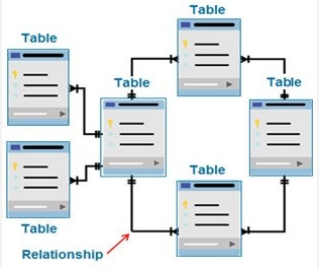
manage

Database

describe

Data model

Relational model
(data processing)

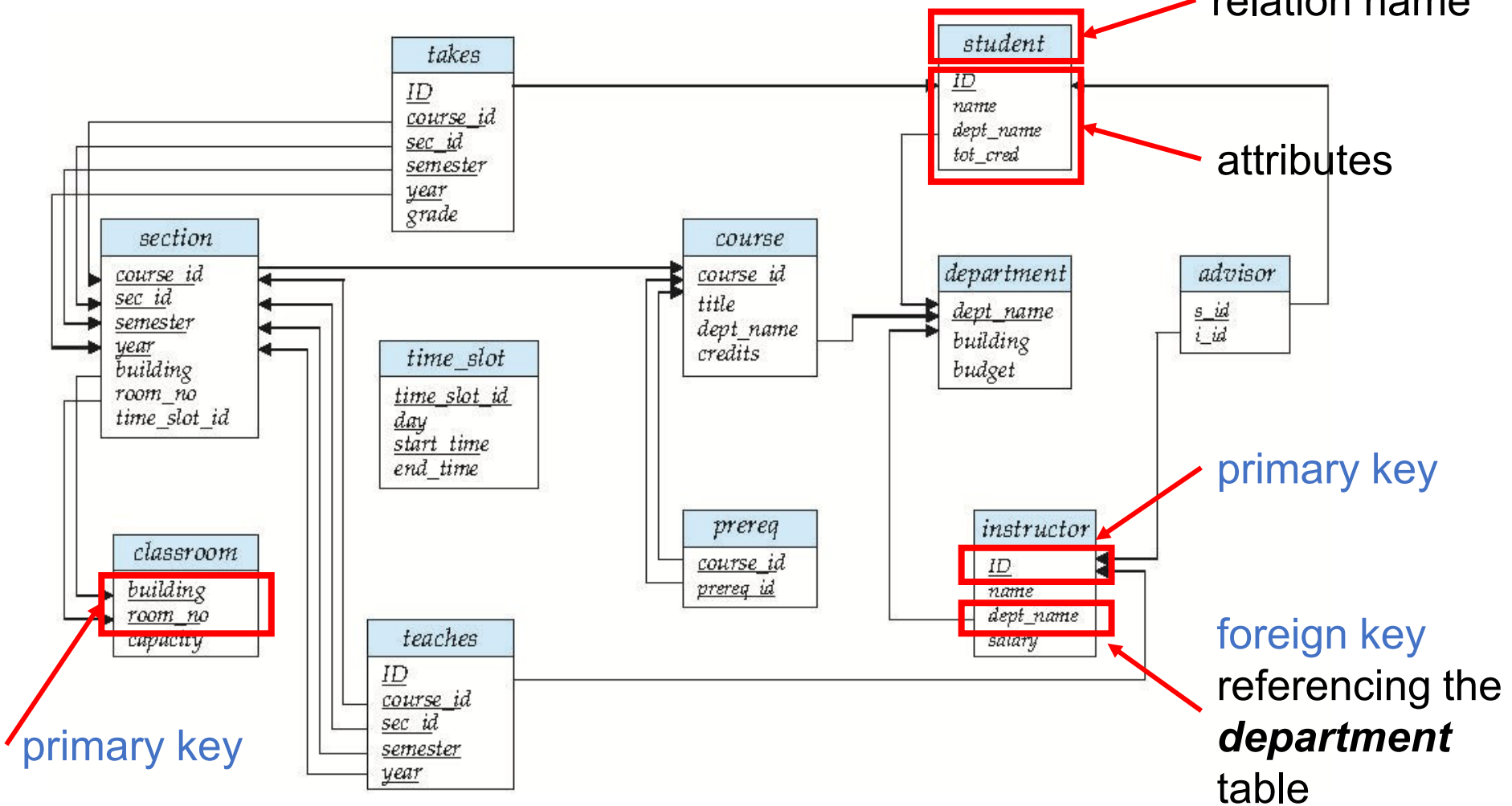


Relational query language

Relational algebra → **SQL**

basis

An Example of a Relational Database



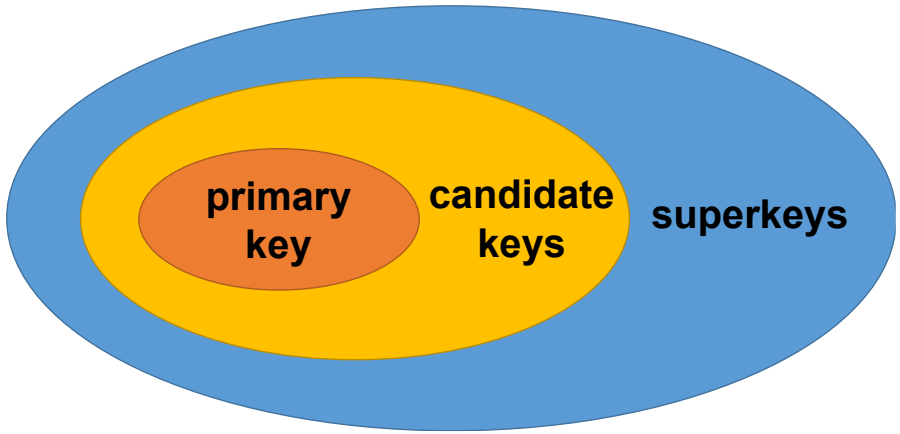
Source: http://www.db-book.com/

Keys



- **Superkey**: A set of one or more attributes that, ***taken collectively***, allows us to unique identify a tuple in a relation.
- **Candidate key**: A set of superkeys for which no proper subset is a superkey.
- **Primary key**: A candidate key chosen by the database designer as the principal means of identifying tuples within a relation.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



Source: http://www.db-book.com/

SQL vs Relational Algebra

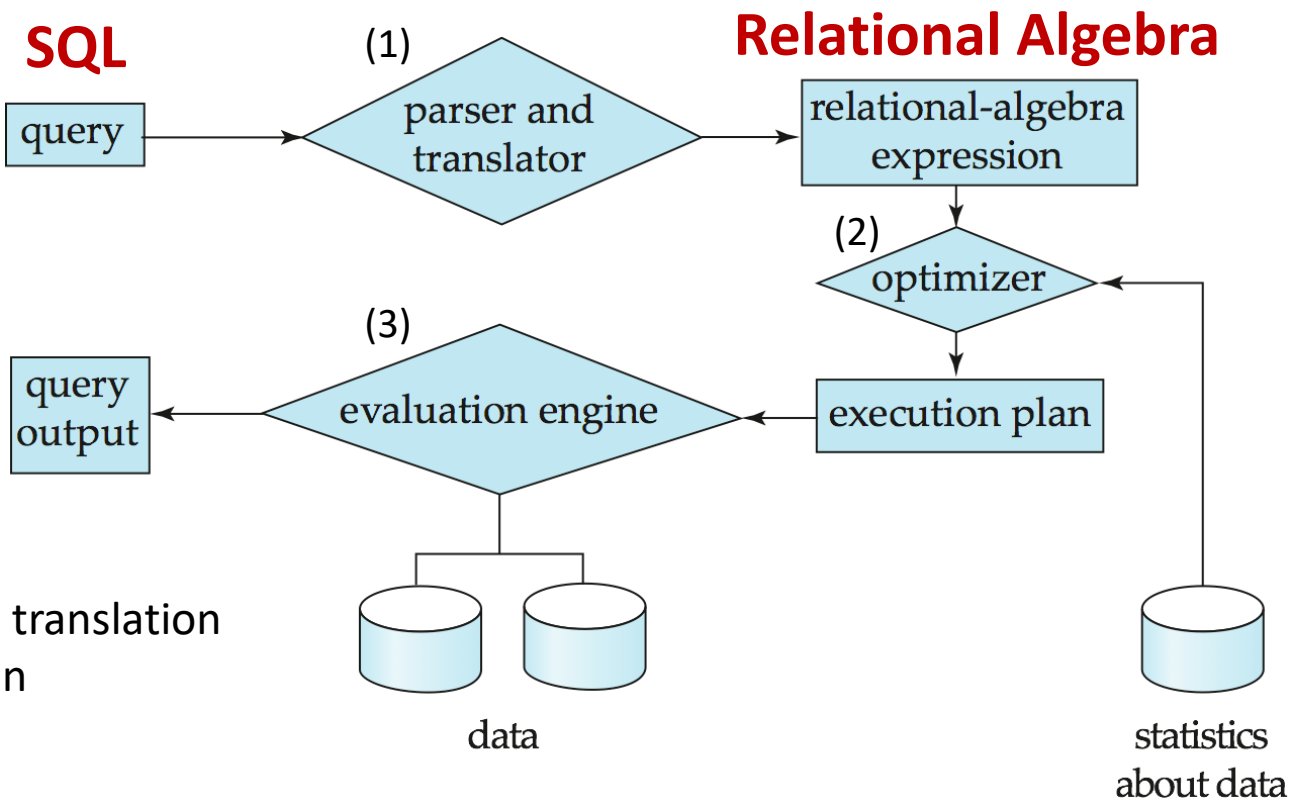
	SQL	Relational Algebra
Function	• Consists of both DDL and DML • The **de facto standard** for accessing database systems today	• Consists of a set of operations used to manipulate and retrieve data • Useful as **internal representation** for query evaluation plans
Type	**Non-procedural** (only specify what data are needed)	**Procedural** (specify both what data are needed and how to get those data)
Based on	Bags (**allow duplicates**)	Sets (**no duplicates**)

Query Processing

select *salary*
from *instructor*
where *salary* < 75000;

$\sigma_{salary<75000}(\Pi_{salary}(instructor))$

$\Pi_{salary}(\sigma_{salary<75000}(instructor))$



1. Parsing and translation
2. Optimization
3. Evaluation

Today's Class

Learning Outcomes of Today's Class



- Understand the basic structure of SQL queries:
  - ***select***, ***from***, ***where***

- Important SQL statements:
  - ***as*** (for rename operation)
  - ***order by, asc, desc*** (for ordering tuples)
  - ***like***, ***ilike*** (for pattern matching)
  - ***union***, ***intersect***, ***except*** (for set operations)
  - ***avg***, ***min***, ***max***, ***sum***, ***count; group by, having*** (for aggregate functions)

Basic Query Structure

Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \ldots, A_n$
from $r_1, r_2, \ldots, r_m$
where P

  - A_i represents an attribute
  - r_i represents a relation
  - P is a predicate.

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- A query takes as its input the relations listed in the **from** clause, operates on them as specified in the **where** and **select** clauses, and then produces a relation as the result.

Some Basic Operations

The Rename Operation

- The SQL allows renaming attributes and relations using the as clause:

  old-name **as** *new-name*

- Reasons that we want to use the rename operation:
  - Two relations in the from clause may have attributes with the same name, in which case an **attribute name is duplicated** in the result.
  - When we use an **arithmetic expression** in the select clause, the resultant attribute does not have a name.
  - When we want to **replace a long relation name** with a shortened version that is more convenient to use elsewhere in the query.
  - When we wish to compare tuples in the **same relation**.

Rename duplicated attribute names

```
select instructor.name, student.name
from instructor, student
where instructor.dept_name = 'Comp. Sci.'
and student.dept_name = 'Comp. Sci.'
```

	name character varying (20)	name character varying (20)
1	Srinivasan	Zhang
2	Srinivasan	Shankar
3	Srinivasan	Williams
4	Srinivasan	Brown
5	Katz	Zhang
6	Katz	Shankar
7	Katz	Williams
8	Katz	Brown
9	Brandt	Zhang
10	Brandt	Shankar
11	Brandt	Williams
12	Brandt	Brown

```
select instructor.name as ins_name,
student.name as stu_name
from instructor, student
where instructor.dept_name = 'Comp. Sci.'
and student.dept_name = 'Comp. Sci.'
```

	ins_name character varying (20)	stu_name character varying (20)
1	Srinivasan	Zhang
2	Srinivasan	Shankar
3	Srinivasan	Williams
4	Srinivasan	Brown
5	Katz	Zhang
6	Katz	Shankar
7	Katz	Williams
8	Katz	Brown
9	Brandt	Zhang
10	Brandt	Shankar
11	Brandt	Williams
12	Brandt	Brown

Rename attributes resulted from arithmetic expressions

select *ID, name, salary/12*
from *instructor*

	id [PK] character varying (5)	name character varying (20)	?column? numeric
1	10101	Srinivasan	5416.6666666666666667
2	12121	Wu	7500.0000000000000000
3	15151	Mozart	3333.3333333333333333
4	22222	Einstein	7916.6666666666666667
5	32343	El Said	5000.0000000000000000
6	33456	Gold	7250.0000000000000000
7	45565	Katz	6250.0000000000000000
8	58583	Califieri	5166.6666666666666667
9	76543	Singh	6666.6666666666666667
10	76766	Crick	6000.0000000000000000
11	83821	Brandt	7666.6666666666666667
12	98345	Kim	6666.6666666666666667

select *ID, name, salary/12*
as *monthly_salary*
from *instructor*

	id [PK] character varying (5)	name character varying (20)	monthly_salary numeric
1	10101	Srinivasan	5416.6666666666666667
2	12121	Wu	7500.0000000000000000
3	15151	Mozart	3333.3333333333333333
4	22222	Einstein	7916.6666666666666667
5	32343	El Said	5000.0000000000000000
6	33456	Gold	7250.0000000000000000
7	45565	Katz	6250.0000000000000000
8	58583	Califieri	5166.6666666666666667
9	76543	Singh	6666.6666666666666667
10	76766	Crick	6000.0000000000000000
11	83821	Brandt	7666.6666666666666667
12	98345	Kim	6666.6666666666666667

Rename relations

- Replace a long relation name with a shortened version

```
select T.name, S.course_id
from instructor as T, teaches as S
where T.ID > S.ID
```

- Compare tuples in the same relation
  - For example: Find the names of all instructors who have a higher salary than at least one instructor in the department of Computer Science

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp. Sci.'
```

Ordering the Display of Tuples

- List in alphabetic order the names of all *instructors*:

select distinct *name*
from *instructor*
order by name

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
  - Example: **order by** *name* **desc**
- Can sort on multiple attributes
  - Example 1: **order by** *dept_name, name*
  - Example 2: **order by** *salary* **desc** , *name* **asc**

```
select *
from instructor
order by name
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	83821	Brandt	Comp. Sci.	92000.00
2	58583	Califieri	History	62000.00
3	76766	Crick	Biology	72000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	98345	Kim	Elec. Eng.	80000.00
9	15151	Mozart	Music	40000.00
10	76543	Singh	Finance	80000.00
11	10101	Srinivasan	Comp. Sci.	65000.00
12	12121	Wu	Finance	90000.00

```
select *
from instructor
order by dept_name, name
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	76766	Crick	Biology	72000.00
2	83821	Brandt	Comp. Sci.	92000.00
3	45565	Katz	Comp. Sci.	75000.00
4	10101	Srinivasan	Comp. Sci.	65000.00
5	98345	Kim	Elec. Eng.	80000.00
6	76543	Singh	Finance	80000.00
7	12121	Wu	Finance	90000.00
8	58583	Califieri	History	62000.00
9	32343	El Said	History	60000.00
10	15151	Mozart	Music	40000.00
11	22222	Einstein	Physics	95000.00
12	33456	Gold	Physics	87000.00

```
select *
from instructor
order by salary desc
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	22222	Einstein	Physics	95000.00
2	83821	Brandt	Comp. Sci.	92000.00
3	12121	Wu	Finance	90000.00
4	33456	Gold	Physics	87000.00
5	76543	Singh	Finance	80000.00
6	98345	Kim	Elec. Eng.	80000.00
7	45565	Katz	Comp. Sci.	75000.00
8	76766	Crick	Biology	72000.00
9	10101	Srinivasan	Comp. Sci.	65000.00
10	58583	Califieri	History	62000.00
11	32343	El Said	History	60000.00
12	15151	Mozart	Music	40000.00

Where Clause Predicates

- SQL includes a **between** comparison operator
- Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq$ \$90,000 and $\leq$ \$100,000)

```
select distinct name
from instructor
where salary between 90000 and 100000
```

- An equivalent but cumbersome expression:

```
select distinct name
from instructor
where salary >= 90000 and salary <= 100000
```

String Operations

String Operations

- SQL includes a string-matching operator for comparisons on character strings.
- The operator **like** uses patterns that are described using two special characters:
  - percent (%). The **%** character matches any substring.
  - underscore (_). The **_** character matches any character.
- Find the names of all instructors whose name starts with 'Br'.

select *name*
from *instructor*
where *name* **like** *'Br%'*

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

select *name*
from *instructor*
where *name* **like** *'Br%'*

Data Output | Explain | Messag

	name character varying (20)
1	Brandt

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

select *name*
from *instructor*
where *name* **like** *'%an'*

Data Output | Explain | Messag

	name character varying (20)
1	Srinivasan

select *name*
from *instructor*
where *name* **like** *'%an%'*

Data Output | Explain | Messag

	name character varying (20)
1	Srinivasan
2	Brandt

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

select *name*
from *instructor*
where *name* **like** *'%ca%'*

select *name*
from *instructor*
where *name* **like** *'%Ca%'*

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

String operations are case sensitive

select *name*
from *instructor*
where *name* **like** *'%ca%'*

Data Output | Explain | Messa

	name character varying (20)

select *name*
from *instructor*
where *name* **like** *'%Ca%'*

Data Output | Explain | Messag

	name character varying (20)
1	Califieri

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

PostgreSQL

Use ILIKE to make match case insensitive

select *name*
from *instructor*
where *name* **ilike** *'%ca%'*

Data Output | Explain | Messag

	name character varying (20)
1	Califieri

select *name*
from *instructor*
where *name* **ilike** *'%s%'*

Data Output | Explain | Messag

	name character varying (20)
1	Srinivasan
2	Einstein
3	El Said
4	Singh

String Operations (Cont.)

- Some more examples:
  - 'Intro%' matches any string beginning with "Intro".
    - For example: "Introduction", "Introvert"
  - '%Comp%' matches any string containing "Comp" as a substring.
    - For example: "Intro. To Computer Science", "Computational Data Science"
  - '_ _ _' matches any string of exactly three characters.
  - '_ _ _ %' matches any string of at least three characters.

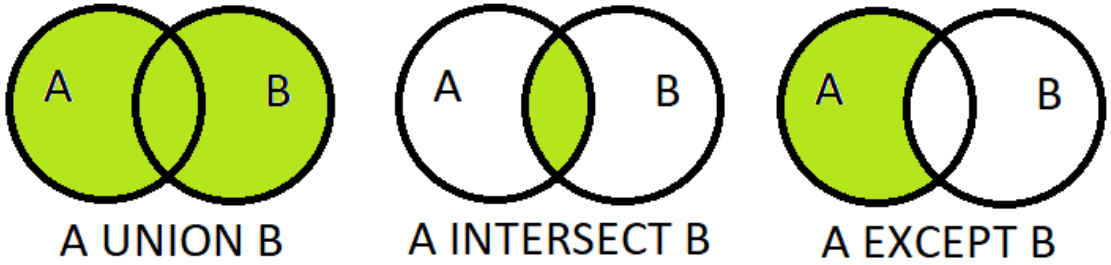
What If We Want to Find Strings With '%' or '_'

- For patterns to include the special pattern character (i.e., % or _), SQL allows the specification of an escape character.
- The **escape** character is used immediately before a special pattern character to indicate that the special pattern is treated like a normal character.
- Consider the following examples, which use a backslash (\) as the escape character:
  - **like** 'ab\%cd%' **escape** '\' matches all strings beginning with "ab%cd"
  - **like** 'ab\\cd%' **escape** '\' matches all strings beginning with "ab\cd"

Set Operations

Set Operations

- The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the mathematical set operations ∪, ∩, and −.



- Find course that ran in Fall 2009 or in Spring 2010:

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
union
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

(**select** * **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
union
(**select** * **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

Data Output | Explain | Messages | Query History

	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)	building character varying (15)	room_number character varying (7)	time_slot_id character varying (4)
1	CS-347	1	Fall	2009	Taylor	3128	A
2	PHY-101	1	Fall	2009	Watson	100	A
3	CS-101	1	Fall	2009	Packard	101	H
4	EE-181	1	Spring	2009	Taylor	3128	C
5	CS-190	1	Spring	2009	Taylor	3128	E
6	CS-190	2	Spring	2009	Taylor	3128	A
7	BIO-101	1	Summer	2009	Painter	514	B
8	CS-319	1	Spring	2010	Watson	100	B
9	CS-319	2	Spring	2010	Taylor	3128	C
10	CS-315	1	Spring	2010	Watson	120	D
11	CS-101	1	Spring	2010	Packard	101	F
12	FIN-201	1	Spring	2010	Packard	101	B
13	HIS-351	1	Spring	2010	Painter	514	C
14	MU-199	1	Spring	2010	Packard	101	D
15	BIO-301	1	Summer	2010	Painter	514	A

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
union
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

Data Output | Explain | Messages | Query History

	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)	building character varying (15)	room_number character varying (7)	time_slot_id character varying (4)
1	CS-347	1	Fall	2009	Taylor	3128	A
2	PHY-101	1	Fall	2009	Watson	100	A
3	CS-101	1	Fall	2009	Packard	101	H
4	EE-181	1	Spring	2009	Taylor		C
5	CS-190	1	Spring	2009	Taylor		E
6	CS-190	2	Spring	2009	Taylor		A
7	BIO-101	1	Summer	2009	Painter		B
8	CS-319	1	Spring	2010	Watson		B
9	CS-319	2	Spring	2010	Taylor		C
10	CS-315	1	Spring	2010	Watson		D
11	CS-101	1	Spring	2010	Packard		F
12	FIN-201	1	Spring	2010	Packard		B
13	HIS-351	1	Spring	2010	Painter		C
14	MU-199	1	Spring	2010	Packard		D
15	BIO-301	1	Summer	2010	Painter		A

Data Output | Explain | Mess

	course_id character varying (8)
1	CS-101
2	CS-315
3	CS-319
4	CS-347
5	FIN-201
6	HIS-351
7	MU-199
8	PHY-101

Set Operations (Cont.)

- Find course that ran in Fall 2009 and in Spring 2010

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
intersect
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

- Find course that ran in Fall 2009 but not in Spring 2010

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
except
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

Data Output | Explain | Messages | Query History

	course_id character varying (8)	semester character varying (6)	year numeric (4)
1	CS-101	Fall	2009
2	CS-347	Fall	2009
3	PHY-101	Fall	2009

select *course_id, semester, year*
from *section*
where *semester* = 'Fall' **and** *year* = 2009

Data Output | Explain | Messages | Query History

	course_id character varying (8)	semester character varying (6)	year numeric (4)
1	CS-101	Spring	2010
2	CS-315	Spring	2010
3	CS-319	Spring	2010
4	CS-319	Spring	2010
5	FIN-201	Spring	2010
6	HIS-351	Spring	2010
7	MU-199	Spring	2010

select *course_id, semester, year*
from *section*
where *semester* = 'Spring' **and** *year* = 2010

Data Output | Explain | Messa

	course_id character varying (8)
1	CS-101

(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2009)
intersect
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2010)

Set Operations (Cont.)

- Find the salaries of all instructors that are less than the largest salary.

select distinct *T.salary*
from *instructor* **as T,** *instructor* **as S**
where *T.salary* < *S.salary*

- Find all salaries of instructors

select distinct *salary*
from *instructor*

- Find the largest salary of all instructors

‘second query’
except
‘first query’

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain

	salary numeric (8,2)
1	87000.00
2	72000.00
3	60000.00
4	90000.00
5	95000.00
6	40000.00
7	75000.00
8	92000.00
9	62000.00
10	65000.00
11	80000.00

select distinct T.salary
from instructor as T

Data Output | Explain

	salary numeric (8,2)
1	87000.00
2	72000.00
3	60000.00
4	90000.00
5	40000.00
6	75000.00
7	92000.00
8	62000.00
9	65000.00
10	80000.00

select distinct T.salary
from instructor as T, instructor as S
where T.salary < S.salary

(select distinct T.salary
from instructor as T)
except
(select distinct T.salary
from instructor as T, instructor as S
where T.salary < S.salary)

Data Output | Explain

	salary numeric (8,2)
1	95000.00

Discussion



- Is there another way to get the largest salary of all instructors?

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

Discussion

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

```
select salary
from instructor
order by salary desc
limit 1
```

Set Operations (Cont.)

- The set operation (**union**, **intersect**, **except**) automatically eliminates duplicates.
- To retain all duplicates use the corresponding multiset versions **union all**, **intersect all** and **except all**.

Set Operations (Discussion)

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
union all
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

Data Output | Explain | Messages | Query History

	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)	building character varying (15)	room_number character varying (7)	time_slot_id character varying (4)
1	CS-347	1	Fall	2009	Taylor		A
2	PHY-101	1	Fall	2009	Watson		A
3	CS-101	1	Fall	2009	Packard		H
4	EE-181	1	Spring	2009	Taylor		C
5	CS-190	1	Spring	2009	Taylor		E
6	CS-190	2	Spring	2009	Taylor		A
7	BIO-101	1	Summer	2009	Painter		B
8	CS-319	1	Spring	2010	Watson		B
9	CS-319	2	Spring	2010	Taylor		C
10	CS-315	1	Spring	2010	Watson		D
11	CS-101	1	Spring	2010	Packard		F
12	FIN-201	1	Spring	2010	Packard		B
13	HIS-351	1	Spring	2010	Painter		C
14	MU-199	1	Spring	2010	Packard		D
15	BIO-301	1	Summer	2010	Painter		A

Data Output | Explain | Messag

	course_id character varying (8)
1	CS-101
2	CS-347
3	PHY-101
4	CS-101
5	CS-315
6	CS-319
7	CS-319
8	FIN-201
9	HIS-351
10	MU-199

Null Values

Null Values

- It is possible for tuples to have a null value, denoted by **null**, for some of their attributes
- **null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving null is null
  - Example: 5 + *null* returns *null*
- The predicate **is null** can be used to check for null values.
  - Example: Find all instructors whose salary is null.

select name
from *instructor*
where *salary* **is null**

Null Values and Three Valued Logic

- Three values – true, false, unknown
- Any comparison with null returns unknown
  - Example: 5 < null or null <> null or null = null
- Three-valued logic using the value unknown:
  - OR: (unknown **or** true) = true,
    (unknown **or** false) = unknown
    (unknown **or** unknown) = unknown
  - AND: *(true* **and** *unknown) = unknown,*
    (false **and** *unknown) = false,*
    (unknown **and** *unknown) = unknown*
  - NOT*: (***not** *unknown) = unknown*
  - "*P* **is unknown**" evaluates to true if predicate *P* evaluates to *unknown*
- Result of where clause predicate is treated as false if it evaluates to unknown

Example

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	12345	Yang	Comp. Sci.	[null]
4	15151	Mozart	Music	40000.00
5	22222	Einstein	Physics	95000.00
6	32343	El Said	History	60000.00
7	33456	Gold	Physics	87000.00
8	45565	Katz	Comp. Sci.	75000.00
9	58583	Califieri	History	62000.00
10	76543	Singh	Finance	80000.00
11	76766	Crick	Biology	72000.00
12	83821	Brandt	Comp. Sci.	92000.00
13	98345	Kim	Elec. Eng.	80000.00

select *
from *instructor*
where *salary* **is null**

Example

select *
from *instructor*
where *salary* **is null**

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	12345	Yang	Comp. Sci.	[null]

select *
from *instructor*
where *salary* **< 60000**

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	15151	Mozart	Music	40000.00

Aggregate Functions

Aggregate Functions

- Aggregate functions operate on the multiset of values of a column of a relation, and return a value

avg: average value
min: minimum value
max: maximum value
sum: sum of values
count: number of values

- Find the average salary of instructors in the computer science department

select avg (*salary*)
from *instructor*
where *dept_name= 'Comp. Sci.';*

Aggregate Functions – Group By

- Find the average salary of instructors in each department

select *dept_name*, **avg** (*salary*) as ave_salary
from *instructor*
group by *dept_name;*

ID	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

instructor table

dept_name	*avg_salary*
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

query result

Aggregate Functions – Group By

- Find the average salary of instructors in each department

select *dept_name*, **avg** (*salary*) as ave_salary
from *instructor*
group by *dept_name;*

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

instructor table

Data Output | Explain | Messages | Query History

	dept_name character varying (20)	ave_salary numeric
1	Biology	72000.000000000000
2	Physics	91000.000000000000
3	Finance	85000.000000000000
4	Music	40000.000000000000
5	History	61000.000000000000
6	Elec. Eng.	80000.000000000000
7	Comp. Sci.	77333.333333333333

query result

Aggregate Functions – Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 80000.

```
select dept_name, avg (salary)
from instructor
group by dept_name
having avg (salary) > 80000
```

- Predicates in the **having** clause are applied after the formation of groups, whereas predicates in the **where** clause are applied before forming groups.

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	dept_name character varying (20)	avg numeric
1	Physics	91000.000000000000
2	Finance	85000.000000000000

select *dept_name*, **avg** (*salary*)
from *instructor*
group by *dept_name*
having avg *(salary) > 80000*

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Note:

- Predicates in the **having** clause are applied after the formation of groups.
- Predicates in the **where** clause are applied before forming groups.

Data Output | Explain | Messages | Query

	dept_name character varying (20)	avg numeric

select *dept_name*, **avg** (*salary*)
from *instructor*
where *salary < 80000*
group by *dept_name*
having avg *(salary) > 80000*

Aggregation With Null Values

- Total all salaries

select sum (*salary*)
from *instructor*

  - Above statement ignores null amounts
- All aggregate operations except **count(*)** ignore tuples with null values on the aggregated attributes.
- What if collection has only null values?
  - Count returns 0
  - All other aggregates return null

Summary



- Understand the basic structure of SQL queries:
  - ***select***, ***from***, ***where***

- Important SQL statements:
  - ***as*** (for rename operation)
  - ***order by, asc, desc*** (for ordering tuples)
  - ***like***, ***ilike*** (for pattern matching)
  - ***union***, ***intersect***, ***except*** (for set operations)
  - ***avg***, ***min***, ***max***, ***sum***, ***count; group by, having*** (for aggregate functions)



Thank you!

michael-yue.yu@polyu.edu.hk

Source: Google Image